Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică
Program: Software Engineering

Operating Systems

Laboratory Work No. 5

The File System with Your Own Hands

Track A: Warm-up on Linux

Elaborated by:
st. gr. FAF-241
Ermicev Sofia

Verified by:
Patricia Reițman

Chișinău – 2026

Contents

1 Introduction **2**

2 Part 1. The Tree and File Types **2**
2.1 Observations 3

3 Part 2. Inodes and Links **3**
3.1 Inode of a regular file 3
3.2 Hard link 3
3.3 Symbolic link 4
3.4 Directory link counts 4
3.5 Logical size versus allocated space 4
3.6 Observations 5

4 Part 3. Permissions **5**
4.1 File permissions 5
4.2 Directory permissions 6
4.3 Special permission bits 7
4.4 Observations 7

5 Part 4. Disks and the Kernel as Files **8**
5.1 Observations 9

6 Conclusion **9**

1. Introduction

The purpose of this laboratory work is to explore the Linux file system directly through command-line experiments. The work focuses on the directory tree and file types, inodes, hard and symbolic links, permissions, special permission bits, disk usage, and the information exposed by the kernel through the `/proc` pseudo-filesystem.

All experiments were performed as the normal user `sofia` in Ubuntu under WSL2. The working directory was `/õs-lab5`. No root privileges were used for the permission experiments.

2. Part 1. The Tree and File Types

The user and working environment were verified first:

```
$ whoami
sofia
```

The root directory and several representative paths were inspected:

```
$ ls -ld /etc /etc/passwd /bin /dev/null /tmp
lrwxrwxrwx  1 root root     7 Apr 22  2024 /bin -> usr/bin
crw-rw-rw-  1 root root  1, 3 Oct  9 02:09 /dev/null
drwxr-xr-x 98 root root  4096 Oct 12 15:40 /etc
-rw-r--r--  1 root root  1545 May  2  2025 /etc/passwd
drwxrwxrwt 13 root root 12288 Oct 12 15:35 /tmp
```

The `file` command confirmed the object types:

```
$ file /usr/bin/ls /etc/passwd /bin
/usr/bin/ls: ELF 64-bit LSB pie executable, x86-64, ...
/etc/passwd: ASCII text
/bin:        symbolic link to usr/bin
```

Disk usage of two system directories was compared:

```
$ du -sh /var/log 2>/dev/null
392M    /var/log
$ du -sh /usr 2>/dev/null
7.4G    /usr
```

2.1. Observations

1. The first character of `ls -l` identifies the file type. `/etc` is a directory (`d`), `/etc/passwd` is a regular file (`-`), `/bin` is a symbolic link (`l`), `/dev/null` is a character device (`c`), and `/tmp` is a directory (`d`).
2. On this system, `/bin` is not a separate directory. It is a symbolic link to `usr/bin`.
3. One character device is `/dev/null`. A block device visible in the system is `/dev/sdc`, which later appears in `lsblk` as the disk containing the root filesystem.
4. `/usr` is much larger than `/var/log`: approximately 7.4 GB versus 392 MB, or roughly nineteen times larger.

3. Part 2. Inodes and Links

3.1. Inode of a regular file

A small file was created and inspected with `stat` and `ls -li`:

```
$ echo hello > notes.txt
$ stat notes.txt
File: notes.txt
Size: 6               Blocks: 8          IO Block: 4096   regular file
Device: 8,32    Inode: 44076       Links: 1
Access: (0644/-rw-r--r--)  Uid: (1000/sofia)  Gid: (1000/sofia)

$ ls -li notes.txt
44076 -rw-r--r-- 1 sofia sofia 6 Oct 12 15:42 notes.txt
```

The inode number was **44076**, and the initial link count was **1**.

3.2. Hard link

A hard link was created:

```
$ ln notes.txt hard.txt
$ ls -li notes.txt hard.txt
44076 -rw-r--r-- 2 sofia sofia 6 Oct 12 15:42 hard.txt
44076 -rw-r--r-- 2 sofia sofia 6 Oct 12 15:42 notes.txt
```

Both names refer to the same inode, and the link count became 2. After removing the original name, the data was still accessible:

```
$ rm notes.txt
$ cat hard.txt
hello
$ ls -li hard.txt
44076 -rw-r--r-- 1 sofia sofia 6 Oct 12 15:42 hard.txt
```

3.3. Symbolic link

A symbolic link was then created:

```
$ ln -s hard.txt soft.txt
$ ls -li hard.txt soft.txt
44076 -rw-r--r-- 1 sofia sofia 6 Oct 12 15:42 hard.txt
44078 lrwxrwxrwx 1 sofia sofia 8 Oct 12 15:43 soft.txt -> hard.txt
$ cat soft.txt
hello
```

After deleting the target, the symbolic link remained but could no longer be followed:

```
$ rm hard.txt
$ cat soft.txt
cat: soft.txt: No such file or directory
$ ls -l soft.txt
lrwxrwxrwx 1 sofia sofia 8 Oct 12 15:43 soft.txt -> hard.txt
```

3.4. Directory link counts

```
$ mkdir d
$ ls -ld d
drwxr-xr-x 2 sofia sofia 4096 Oct 12 15:43 d

$ mkdir d/x d/y
$ ls -ld d
drwxr-xr-x 4 sofia sofia 4096 Oct 12 15:43 d
```

3.5. Logical size versus allocated space

```
$ echo x > one.txt
$ ls -l one.txt
```

```
-rw-r--r-- 1 sofia sofia 2 Oct 12 15:43 one.txt
$ du -h one.txt
4.0K    one.txt

$ truncate -s 1G big.img
$ ls -lh big.img
-rw-r--r-- 1 sofia sofia 1.0G Oct 12 15:43 big.img
$ du -h big.img
0       big.img
```

3.6. Observations

1. In `stat`, `Inode: 44076` is the inode number, while `Links: 1` is the number of directory entries (hard links) referring to that inode.
2. Removing `notes.txt` did not remove the data because `hard.txt` still referred to the same inode. Therefore, `rm` removes a directory entry (a name); file data is released only when no hard links and no open references remain.
3. `soft.txt` is a separate inode containing the pathname `hard.txt`. When the target was removed, the symbolic link became broken. Its size is 8 bytes because the stored target string `hard.txt` contains eight characters.
4. A new directory has link count 2: one entry from its parent and its own `.` entry. Each subdirectory adds one more reference to the parent through its `..` entry, so adding `x` and `y` increased the count to 4.
5. The 2-byte file occupies 4.0K because disk space is allocated in filesystem blocks. The 1 GB file created with `truncate` occupied 0 blocks because it was a sparse file: its logical size was large, but no data blocks had yet been written.

4. Part 3. Permissions

4.1. File permissions

```
$ echo secret > s.txt
$ chmod 600 s.txt
$ ls -l s.txt
-rw------- 1 sofia sofia 7 Oct 12 15:47 s.txt
```

```
$ chmod 000 s.txt
$ cat s.txt
cat: s.txt: Permission denied

$ chmod u+r s.txt
$ cat s.txt
secret
```

The current umask and permissions of newly created objects were:

```
$ umask
0022
$ touch new.txt
$ mkdir newdir
$ ls -ld new.txt newdir
-rw-r--r-- 1 sofia sofia    0 Oct 12 15:53 new.txt
drwxr-xr-x 2 sofia sofia 4096 Oct 12 15:53 newdir
```

4.2. Directory permissions

A directory was tested with read-only and execute-only permissions for the owner:

```
$ mkdir box
$ echo hi > box/f
$ chmod u=r,go= box
$ ls box
ls: cannot access 'box/f': Permission denied
f
$ cat box/f
cat: box/f: Permission denied
```

With execute permission only:

```
$ chmod u=x,go= box
$ ls box
ls: cannot open directory 'box': Permission denied
$ cat box/f
hi
```

This shows that read permission on a directory allows listing its entry names, while execute permission allows pathname traversal and access to a known entry.

4.3. Special permission bits

```
$ ls -l /usr/bin/passwd
-rwsr-xr-x 1 root root 64152 May 30 2024 /usr/bin/passwd

$ ls -ld /tmp
drwxrwxrwt 13 root root 12288 Oct 12 15:35 /tmp

$ cat /etc/shadow
cat: /etc/shadow: Permission denied

$ ls -l /etc/shadow
-rw-r----- 1 root shadow 841 May 2 2025 /etc/shadow
```

4.4. Observations

1. Common numeric modes are:
    - `600 = rw-------`: private file readable and writable only by its owner, for example a sensitive configuration file.
    - `644 = rw-r--r--`: owner may edit; everyone may read, common for ordinary text files.
    - `755 = rwxr-xr-x`: owner may modify and execute; others may read and execute, common for programs and public directories.
    - `700 = rwx------`: private directory or executable accessible only by its owner.
2. Ownership does not automatically bypass permission bits. With mode `000`, the owner is still subject to the file's access mode and therefore cannot read it as a normal user.
3. The umask was `0022`. New regular files start from a maximum of `666`; removing the masked bits gives `644`. New directories start from `777`; applying the same mask gives `755`.
4. For directories, `r` permits reading the directory entries (listing names), whereas `x` permits traversal/search of the directory and access to a known pathname inside it.
5. The `s` in `/usr/bin/passwd` is the set-user-ID bit. It allows the program to run with the effective user ID of its owner (root), which is necessary for controlled updates to protected password data. The `t` in `/tmp` is the

sticky bit; it prevents users from deleting or renaming files owned by other users in this shared writable directory.

5. Part 4. Disks and the Kernel as Files

The block devices were inspected with `lsblk`:

```
NAME MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
sda    8:0    0 388.4M  1 disk
sdb    8:16   0     4G  0 disk [SWAP]
sdc    8:32   0     1T  0 disk /mnt/wslg/distro
                               /
```

The root filesystem usage was:

```
$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/sdc       1007G   22G  934G   3% /
```

Its filesystem type was identified through the mount table:

```
$ mount | grep " / "
/dev/sdc on / type ext4 (rw,relatime,discard,errors=remount-ro,data=
   ordered)
```

CPU and memory information were read from `/proc`:

```
$ grep -c processor /proc/cpuinfo
16

$ head -3 /proc/meminfo
MemTotal:       14186832 kB
MemFree:        13273344 kB
MemAvailable:   13354732 kB
```

The status of the process reading `/proc/self/status` was:

```
$ head -5 /proc/self/status
Name:   head
Umask:  0022
State:  R (running)
Tgid:   69035
Ngid:   0
```

Open file descriptors were also visible as symbolic links:

```
$ ls -l /proc/self/fd
total 0
lrwx------ 1 sofia sofia 64 Oct 12 15:54 0 -> /dev/pts/2
lrwx------ 1 sofia sofia 64 Oct 12 15:54 1 -> /dev/pts/2
lrwx------ 1 sofia sofia 64 Oct 12 15:54 2 -> /dev/pts/2
lr-x------ 1 sofia sofia 64 Oct 12 15:54 3 -> /proc/68897/fd
```

5.1. Observations

1. The WSL2 Linux root filesystem is on the block device `/dev/sdc`. It is mounted at `/` using the `ext4` filesystem. The system also exposes `/dev/sdb` as 4 GB of swap.

2. The root filesystem has a size of approximately 1007 GB, with about 22 GB used and 934 GB available. It was approximately 3% full at the time of the experiment.

3. According to `/proc/cpuinfo`, the environment exposes 16 logical processors. `/proc/meminfo` reports 14,186,832 kB of RAM, approximately 13.5 GiB.

4. `/proc/self/status` showed the process name `head`. The special `/proc/self` path refers to the process that is currently accessing it; in the command `head -5 /proc/self/status`, that process is `head` itself.

6. Conclusion

This laboratory work demonstrated the main Linux file-system concepts directly through observation. File names were shown to be directory entries that refer to inodes, while hard links create additional names for the same inode and symbolic links store a pathname to another object. The experiments also demonstrated that logical file size and allocated disk space are different concepts, especially for sparse files.

The permission experiments showed how read, write, and execute bits behave differently for files and directories, how `umask` influences newly created objects, and why setuid and sticky bits are necessary for common system facilities such as `passwd` and `/tmp`. Finally, the disk and `/proc` experiments connected the filesystem view with storage devices, CPU and memory information, processes, and open file descriptors. Overall, the work showed how Linux exposes both persistent storage and many kernel-managed resources through a unified file-oriented interface.